



Fig. 3: Using only two mouse-clicks, the analyser tool detects different types of code coverage

Statement or block coverage. This measures how many of the C-statements or code blocks have been executed during a test session. It does not measure how branches affect which C-statements or code blocks are executed.

Function coverage. This only measures which or how many of the C-functions have been called during a test session. It does not measure which or how many of the function calls in a code section are actually executed, or the quality of the testing of the function itself.

Function call coverage. This measures which or how many of the function calls in a code section have actually been made during a test session.

Branch coverage. This measures whether all code blocks and alternate branch paths have been executed in a code section (such as both the 'if' and 'else' part in an if-else statement). Branch coverage typically requires a code section to be executed a sufficient number of times, so that all alternative branch directions are tested. As all branch paths must be executed, all corresponding code blocks are executed as well.

Modified condition/decision coverage. This is a very advanced type of code-coverage analysis, which is often used for measuring the test quality of safety-critical software—generally in

flight-control and aerospace applications. However, many other projects can also benefit from this advanced test analysis.

This kind of code-coverage is applied to applications through which the highest reliability is expected. It extends branch coverage with the additional requirement that all sub-expressions in complex decisions (such as in a complex if-statement) must drive the branch decision independently of the other sub-expressions.

Until recently, there have been few test-quality measurement tools available. These are limited by one or more of the following problems:

1. Testing using weak types of code-coverage analysis only
2. Testing in PC environments and not on the embedded target
3. Very expensive
4. Difficult to use
5. Lack of integration with other embedded development tools

New tools are highly integrated supporting a wide range of code-coverage analysis types. One of their most powerful features is that the analysis is conducted on the embedded target. Connecting to the target via a JTAG probe, the embedded application is run directly on the target board with the code-coverage information recorded for later inspection and quantitative analysis.

As an example of these tools' capabilities, Fig. 2 shows a trivial code section that contains three blocks—a red code block, which is always executed; a green block, sometimes executed depending on the branch decision made in the if-statement; and a blue block, which again is always executed.

The branch decision in the if-statement drives the selection of one out of the two possible execution paths—either directly from the red to the blue code block; or passing from red to blue via the green code block.

Using only a few mouse-clicks, the easy-to-use in-target analyser tools can detect all the different types of code coverage including statement/block coverage, function coverage, function call coverage, branch coverage and modified condition/decision coverage.

Both the verification and analysis tools are integrated in the integrated development environment. It's this new generation of advanced embedded test tools that will truly enable software developers to be confident of delivering high-quality software, making a significant contribution to new products that work first time. ●

The authors are from Atollic—a provider of advanced development tools and expert services within the field of embedded systems and technical PC applications. Article shared by Embedded Systems Solutions, Bengaluru—Atollic's authorised distributor in India